Primal–Dual Subgradient and Dual Averaging Experiments

1 Introduction

This report studies primal–dual subgradient methods for convex nonsmooth optimization, with a focus on Nesterov's dual averaging framework [1]. The basic problem is

$$\min_{x \in Q} f(x),$$

where Q is a closed convex set and f is convex but not necessarily differentiable. In such problems the oracle returns a subgradient $g \in \partial f(x)$. Unlike gradients of smooth objectives, subgradients do not generally vanish near an optimum. This is the main reason classical subgradient methods need decreasing primal step sizes. At the same time, the past subgradients define useful global affine lower models of the objective, so discarding their information too quickly can be inefficient.

Dual averaging separates these two roles. The method generates test points $x_0, x_1, \ldots$. At each point x_i, the oracle returns a subgradient $g_i \in \partial f(x_i)$, and the algorithm assigns this subgradient a nonnegative averaging weight λ_i. The weighted average of past test points is denoted by $\hat{x}_k$. The method accumulates subgradients in a dual variable and obtains the next iterate x_{k+1} by regularizing this accumulated linear model with a prox function.

Let $d : Q \to \mathbb{R}$ be a strongly convex prox function with prox-center

$$x_0 = \arg\min_{x \in Q} d(x), \qquad d(x_0) = 0.$$

The implementation starts from this prox-center, so the same x_0 is also the initial test point. The parameter $D > 0$ defines the prox ball

$$F_D = \{x \in Q : \ d(x) \le D\}.$$

In unrestricted SDA runs, D is used in the stopping certificate but the iterate x_k may lie outside F_D. In restricted runs, the implementation also projects iterates back to F_D.

The associated prox mapping is

$$\pi_\beta(s) = \arg\min_{x \in Q} \{-\langle s, x\rangle + \beta d(x)\}, \qquad \beta > 0.$$

For the Euclidean prox $d(x) = \frac{1}{2}\|x - x_0\|_2^2$, this reduces in the unconstrained case to $\pi_\beta(s) = x_0 + s/\beta$.

The generic dual averaging update is

$$s_{k+1} = s_k + \lambda_k g_k, \qquad x_{k+1} = \pi_{\beta_{k+1}}(-s_{k+1}),$$

where s_k is the accumulated dual vector and $\beta_{k+1} > 0$ controls the strength of the prox regularization.

The primal–dual feature of the method is the computable certificate

$$\delta_k(D) = \max_{x \in F_D} \sum_{i=0}^{k} \lambda_i \langle g_i, x_i - x\rangle, \qquad S_k = \sum_{i=0}^{k} \lambda_i.$$

If the selected radius satisfies the required condition

$$D \geq d(x^\star),$$

then $x^\star \in F_D$ and the normalized primal–dual certificate gap $\delta_k(D)/S_k$ is an upper bound on the objective error of the averaged iterate $\hat{x}_k$. This radius condition is essential: when $x^\star \notin F_D$, the certificate is no longer a valid certificate for the original problem and can even become negative. The experiments therefore distinguish the primal–dual certificate gap from the benchmark objective gap $f(\hat{x}_k) - f^\star$ and from the lasso train-objective gap $F(\hat{x}_k) - F(x_{\text{saga}})$.

The implemented methods are simple dual averaging (SDA), weighted SDA, stochastic simple averages (SSA), and a projected subgradient baseline. The deterministic benchmark experiments use nonsmooth convex objectives with known optima, so their objective gap is $f(\hat{x}_k) - f^\star$. The lasso logistic experiments use empirical train objectives and compare against a matched `sklearn` SAGA reference; their reported train-objective gap is $F(\hat{x}_k) - F(x_{\text{saga}})$. This distinction is used throughout the report.

2 Implemented Methods

2.1 Shared Dual Averaging State

This section describes the implementation using the same notation as the code. The common inputs are an initial point x_0, a prox-radius parameter D, a maximum iteration count, a subgradient oracle G, a step multiplier γ_{mult} for the SDA variants, and, for deterministic SDA, a stopping tolerance ε. The boolean option `restrict_to_fd` controls whether each new iterate is projected back to F_D. If this option is false, D is still used in the primal–dual certificate gap, but it does not constrain the iterates.

Symbol	Meaning
x_k	primal test point at iteration k
g_k	subgradient returned at x_k, usually $g_k \in \partial f(x_k)$
λ_k	dual averaging weight assigned to g_k
s_k	accumulated dual vector
S_k	accumulated averaging weight
$\hat{x}_k$	weighted average of past primal test points
D	prox-radius parameter defining $F_D = \{x : d(x) \leq D\}$
$\delta_k(D)/S_{k+1}$	normalized primal–dual certificate gap used for deterministic SDA stopping
γ	prox scale used in $B_{k+1} = \gamma\hat{\beta}_{k+1}$
γ_{mult}	dimensionless multiplier applied to the baseline $\gamma^\star$

The implementation stores the state after processing iteration k at index $k+1$. Thus the denominator S_{k+1} below contains the weights $\lambda_0, \ldots, \lambda_k$, which is the same accumulated quantity denoted by $S_k = \sum_{i=0}^{k} \lambda_i$ in the introductory certificate formula.

All dual averaging variants maintain a sequence of primal test points $\{x_k\}$, subgradients $g_k \in \partial f(x_k)$, nonnegative weights λ_k, the accumulated dual vector

$$s_{k+1} = s_k + \lambda_k g_k, \qquad s_0 = 0,$$

and the accumulated weight

$$S_{k+1} = S_k + \lambda_k, \qquad S_0 = 0.$$

The averaged primal point is updated as

$$\hat{x}_{k+1} = \frac{S_k \hat{x}_k + \lambda_k x_k}{S_{k+1}}.$$

The implementation also stores

$$\delta_k(D) = \sum_{i=0}^{k} \lambda_i \langle g_i, x_i - x_0 \rangle + \xi_D(-s_{k+1}),$$

where

$$\xi_D(s) = \max_{x \in F_D} \langle s, x - x_0 \rangle.$$

The quantity $\delta_k(D)/S_{k+1}$ is the primal–dual certificate gap used for deterministic SDA stopping. It is not the same metric as the benchmark objective gap or the lasso train-objective gap to SAGA.

The code implements three prox geometries. For the Euclidean prox,

$$d(x) = \frac{1}{2}\|x - x_0\|_2^2, \qquad \xi_D(s) = \sqrt{2D}\|s\|_2.$$

For the weighted Euclidean prox with positive diagonal weights h_i,

$$d(x) = \frac{1}{2}\sum_i h_i (x_i - x_{0,i})^2, \qquad \xi_D(s) = \sqrt{2D \sum_i s_i^2 / h_i}.$$

For the entropy prox on the probability simplex,

$$d(x) = \log n + \sum_{i=1}^{n} x_i \log x_i, \qquad x_0 = (1/n, \ldots, 1/n).$$

All implemented prox functions are treated as having strong convexity parameter $\sigma = 1$. Nesterov's SDA bound contains the expression

$$\gamma D + \frac{L^2}{2\sigma\gamma}.$$

Balancing these two terms gives the baseline scale

$$\gamma^\star = \frac{L}{\sqrt{2\sigma D}} = \frac{L}{\sqrt{2D}},$$

where L is the Lipschitz estimate supplied by the objective wrapper. The implementation then uses

$$\gamma = \gamma^\star \gamma_{\text{mult}}.$$

The multiplier γ_{mult} is dimensionless and is used for tuning around the theoretical baseline. Larger γ increases the prox regularization and makes the accumulated subgradient model move the iterate less aggressively; smaller γ weakens this regularization and produces larger primal moves.

2.2 Simple Dual Averaging

Simple Dual Averaging uses unit aggregation weights,

$$\lambda_k = 1.$$

The scale sequence follows Nesterov's recurrence

$$\hat{\beta}_0 = \hat{\beta}_1 = 1, \qquad \hat{\beta}_{k+1} = \hat{\beta}_k + \frac{1}{\hat{\beta}_k},$$

and the implementation sets

$$B_{k+1} = \gamma\hat{\beta}_{k+1}.$$

Here B_{k+1} is the implementation name for the prox parameter β_{k+1} in the mapping $\pi_{\beta_{k+1}}$. The primal update is

$$x_{k+1} = \pi_{B_{k+1}}(-s_{k+1}).$$

For unconstrained Euclidean runs this is

$$x_{k+1} = x_0 - \frac{s_{k+1}}{B_{k+1}}.$$

Pseudocode.

1. Choose x_0, D, ε, γ_{mult}, and a maximum iteration count. Set $s_0 = 0$, $S_0 = 0$, and $\hat{x}_0 = x_0$. Set $\hat{\beta}_0 = \hat{\beta}_1 = 1$.
2. Set $\gamma = \gamma^\star\gamma_{\mathrm{mult}}$, where $\gamma^\star = L/\sqrt{2D}$.
3. For $k = 0, 1, \ldots$:
   (a) compute $g_k \in \partial f(x_k)$;
   (b) set $\lambda_k = 1$, $s_{k+1} = s_k + g_k$, and $S_{k+1} = S_k + 1$;
   (c) use $\hat{\beta}_1 = 1$ for $k = 0$, and for later iterations update $\hat{\beta}_{k+1} = \hat{\beta}_k + 1/\hat{\beta}_k$; set $B_{k+1} = \gamma\hat{\beta}_{k+1}$, and compute $x_{k+1} = \pi_{B_{k+1}}(-s_{k+1})$;
   (d) optionally project x_{k+1} to F_D in restricted runs;
   (e) update $\hat{x}_{k+1}$ and compute $\delta_k(D)/S_{k+1}$;
   (f) stop if the deterministic primal–dual certificate gap is at most ε.

2.3 Weighted Dual Averaging

Weighted SDA uses the same primal update and the same $\hat{\beta}_k$ sequence as simple SDA, but changes the aggregation weights to

$$\lambda_k = \begin{cases} 1/\|g_k\|_*, & \|g_k\|_* > 0, \\ 0, & \|g_k\|_* = 0. \end{cases}$$

This gives each nonzero subgradient a unit dual-norm contribution in the dual aggregate, since $\|\lambda_k g_k\|_* = 1$. The zero-subgradient convention prevents division by zero. Nesterov's weighted dual averages method is parameterized by ρ, whereas the implementation exposes the same γ interface used by SDA. Since the implemented prox functions use $\sigma = 1$, the code treats

$$\rho = \frac{1}{\gamma}, \qquad \gamma = \gamma^\star\gamma_{\mathrm{mult}}.$$

Thus γ_{mult} still controls the practical scale of the prox regularization, but the weighted method should be interpreted through the paper's inverse ρ convention.

Pseudocode.

1. Initialize x_0, $s_0 = 0$, $S_0 = 0$, $\hat{x}_0 = x_0$, and $\gamma = \gamma^\star\gamma_{\mathrm{mult}}$. Set $\hat{\beta}_0 = \hat{\beta}_1 = 1$.
2. For $k = 0, 1, \ldots$:
   (a) compute $g_k \in \partial f(x_k)$;

(b) compute $\lambda_k = 1/\|g_k\|_*$, using $\lambda_k = 0$ if $g_k = 0$;

(c) update $s_{k+1} = s_k + \lambda_k g_k$ and $S_{k+1} = S_k + \lambda_k$;

(d) use the same $\hat{\beta}_k$ recursion as simple SDA and compute $x_{k+1} = \pi_{\gamma\hat{\beta}_{k+1}}(-s_{k+1})$;

(e) optionally project x_{k+1} to F_D in restricted runs;

(f) update $\hat{x}_{k+1}$ using the weighted average and compute the primal–dual certificate gap;

(g) stop if the deterministic primal–dual certificate gap is at most ε.

2.4 Stochastic Simple Averages

SSA is the stochastic version of simple dual averaging. Instead of evaluating a full deterministic subgradient, the method draws an independent random sample ξ_k and uses

$$\tilde{g}_k = G(x_k, \xi_k).$$

For the finite-sum lasso logistic objective, ξ_k is a training-row index sampled with replacement. The experiments use batch size one, so the sampled logistic gradient for the feature coordinates is

$$a_{\xi_k}\left(\sigma(a_{\xi_k}^{\mathsf{T}} w + b) - y_{\xi_k}\right),$$

where a_{ξ_k} is the sampled feature vector. The intercept coordinate is

$$\sigma(a_{\xi_k}^{\mathsf{T}} w + b) - y_{\xi_k}.$$

The same deterministic lasso subgradient term is then added using the λ/n scaling of the full objective, where λ is the lasso regularization parameter and is unrelated to the dual averaging weights λ_k. The sampled logistic gradient is an unbiased estimator of the empirical logistic gradient component, while the lasso term is deterministic at the current iterate. The SSA implementation uses the SDA recursion with $\lambda_k = 1$, but disables stopping on the sampled primal–dual certificate gap because that per-run quantity is noisy and is not the deterministic certificate used by full-gradient SDA. The sampled certificate is still tracked internally for diagnostics.

Pseudocode.

1. Initialize x_0, $s_0 = 0$, $S_0 = 0$, $\hat{x}_0 = x_0$, random seed, and $\gamma = \gamma^\star \gamma_{\text{mult}}$. Set $\hat{\beta}_0 = \hat{\beta}_1 = 1$.
2. For $k = 0, 1, \ldots, \texttt{max\_iter} - 1$:
   (a) sample a training index or mini-batch ξ_k;
   (b) compute the stochastic subgradient $\tilde{g}_k = G(x_k, \xi_k)$;
   (c) set $s_{k+1} = s_k + \tilde{g}_k$, $S_{k+1} = S_k + 1$;
   (d) use the same $\hat{\beta}_k$ recursion as simple SDA and compute $x_{k+1} = \pi_{\gamma\hat{\beta}_{k+1}}(-s_{k+1})$;
   (e) update the simple average $\hat{x}_{k+1}$.

2.5 Projected Subgradient Baseline

The baseline method is Euclidean projected subgradient descent. It uses

$$\alpha_k = \frac{\alpha}{\sqrt{k+1}}, \qquad x_{k+1} = x_k - \alpha_k g_k.$$

When a restricted run is requested, x_{k+1} is projected to the Euclidean prox ball $F_D = \{x : \frac{1}{2}\|x - x_0\|_2^2 \le D\}$. The method tracks the same averaged iterate $\hat{x}_k$ for metric reporting, but it does not compute a dual averaging primal–dual certificate gap.

Pseudocode.

1. Choose x_0, α, D, and `max_iter`.
2. For $k = 0, 1, \ldots,$ `max_iter` $- 1$:
   (a) compute $g_k \in \partial f(x_k)$;
   (b) set $\alpha_k = \alpha/\sqrt{k+1}$;
   (c) update $x_{k+1} = x_k - \alpha_k g_k$;
   (d) optionally project x_{k+1} to F_D;
   (e) update the arithmetic average $\hat{x}_{k+1}$.

2.6 Objective Definitions

The nonsmooth benchmark registry contains convex objectives with known reference optima. The Euclidean benchmark grid uses the nine objectives in Table 1; the simplex entropy experiment uses the three objectives in Table 2. The deterministic subgradient convention at nondifferentiable ties is fixed by the implementation: absolute-value terms use sign zero at exactly zero, and max-affine objectives select one active affine piece.

Objective id	Definition	Parameters used
`abs_a2`	$f(x) = \lvert x - a\rvert$	$a = 2$.
`weighted_l1_shift_1d`	$f(x) = \sum_{j=1}^{3} w_j \lvert x - a_j\rvert$	$w = (1.25, 0.75, 0.5)$, $a = (2, -1.5, 3.5)$.
`max_affine_1d`	$f(x) = \max_j \{m_j x + b_j\}$	$m = (1.2, -0.8, 0.35)$, $b = (-2, 2.6, 0.9)$.
`weighted_l1_shift_2d`	$f(x) = \sum_{i=1}^{2} w_i \lvert x_i - a_i\rvert$	$w = (1, 1.5)$, $a = (2, -1)$.
`linf_shift_2d`	$f(x) = \max_i \lvert x_i - a_i\rvert$	$a = (1.5, -2.5)$.
`weighted_l1_shift_4d`	$f(x) = \sum_{i=1}^{4} w_i \lvert x_i - a_i\rvert$	$w = (1, 0.5, 1.25, 0.75)$, $a = (2.5, -1.5, 1, 3)$.
`max_affine_4d`	$f(x) = \max_j \{a_j^\mathsf{T} x + b_j\}$	Rows $a_j = e_1, e_2, e_3, e_4, -\mathbf{1}$, $b = (-2, 1.5, -0.75, -2.5, 3.75)$.
`ill_conditioned_l1_shift_8d`	$f(x) = \sum_{i=1}^{8} w_i \lvert x_i - a_i\rvert$	$w = (0.1, 0.2, 0.5, 1, 2, 5, 10, 25)$, $a = (2, -1.5, 1, 3, -2, 0.5, 1.5, -0.75)$.
`ill_conditioned_max_affine_8d`	$f(x) = \max_i c_i \lvert x_i - a_i\rvert$	$c = (0.1, 0.2, 0.5, 1, 2, 5, 10, 25)$, $a = (2, -1.5, 1, 3, -2, 0.5, 1.5, -0.75)$.

Table 1: Nonsmooth Euclidean benchmark objectives.

The simplex objectives are defined on

$$\Delta_n = \{x \in \mathbb{R}^n : \; x_i \geq 0, \; \sum_i x_i = 1\}.$$

For max-affine simplex objectives, the objective is

$$f(x) = \max_j \{a_j^\mathsf{T} x + b_j\}.$$

For logistic regression, let $a_i \in \mathbb{R}^p$ be the feature vector and $y_i \in \{0, 1\}$ the binary label. The lasso logistic objective used in the experiments is

$$F(w, b) = \frac{1}{n} \sum_{i=1}^{n} \Big[\log(1 + \exp(a_i^\mathsf{T} w + b)) - y_i (a_i^\mathsf{T} w + b)\Big] + \frac{\lambda}{n} \|w\|_1.$$

The intercept b is appended as the final coordinate of the optimization vector and is not penalized. Here λ denotes the lasso regularization parameter, not a dual averaging weight. The implementation first loads numeric CSV features with binary labels, creates a stratified train/test split, standardizes features using train-set statistics only, and then appends a bias column.

Objective id	Definition	Parameters used
`simplex_linear_8d`	$f(x) = c^\top x$, $x \in \Delta_8$	$c = (0.8, -0.4, 1.2, 0.1, -0.9, 0.3, 0.6, -0.2)$.
`simplex_max_affine_8d`	$f(x) = \max_{1 \le j \le 6}\{a_j^\top x + b_j\}$, $x \in \Delta_8$	$b = (0, 0.1, -0.05, 0.2, -0.1, 0.05)$. The six rows a_j are $(1, -0.5, 0.2, 0, -0.3, 0.4, 0.1, -0.2)$, $(-0.4, 0.8, -0.1, 0.3, 0.2, -0.5, 0.6, 0)$, $(0.2, 0.1, 0.9, -0.4, 0.5, 0, -0.3, 0.4)$, $(0, -0.2, 0.4, 1, -0.5, 0.3, 0.2, -0.1)$, $(-0.3, 0.4, 0, -0.2, 0.9, -0.1, 0.5, 0.2)$, $(0.5, 0, -0.4, 0.2, 0.1, 0.8, -0.2, 0.3)$.
`simplex_sparse_max_affine_16d`	$f(x) = \max_{1 \le j \le 16}\{a_j^\top x + b_j\}$, $x \in \Delta_{16}$	$b_j = 0.02(j-1)$. For $j = 1, \dots, 8$, row a_j has 1 in coordinate j, -0.2 in coordinate $j+1$, and zeros elsewhere. For $j = 9, \dots, 16$, row a_j has -0.5 in coordinate $j-8$, 0.7 in coordinate $j-4$, and zeros elsewhere.

Table 2: Simplex benchmark objectives used with entropy prox.

2.7 Logistic Regression Datasets

The synthetic datasets were generated by `data/generate_logistic_data.py`. The generator takes the number of samples, feature dimension, coefficient vector β, intercept b, random seed, label-flip probability, and output path. It samples features

$$a_i \sim N(0, I),$$

forms logits $a_i^\top \beta + b$, samples $y_i \sim \text{Bernoulli}(\sigma(a_i^\top \beta + b))$, and then flips each label independently with probability p_{flip}. The recorded synthetic parameters are shown in Table 4. The two larger synthetic datasets are included to stress the weighted SDA versus SSA comparison on larger finite-sum lasso objectives.

The public datasets are downloaded and normalized by the script `data/download_binary_datasets.py`. They all use the same normalized schema: numeric feature columns and a binary label column. The Iris dataset is converted to Setosa versus non-Setosa classification. The Wisconsin Diagnostic Breast Cancer dataset classifies malignant versus benign tumors. The Banknote Authentication dataset classifies genuine versus forged banknotes from image features. The Spambase dataset classifies email as spam or non-spam.

Dataset	Type	Samples	Features	Description
`iris.csv`	UCI public	150	4	Setosa versus non-Setosa Iris classification
`breast_cancer_wdbc.csv`	UCI public	569	30	malignant versus benign tumor classification
`banknote_authentication.csv`	UCI public	1372	4	genuine versus forged banknote classification
`spambase.csv`	UCI public	4601	57	spam versus non-spam email classification
`synthetic_logistic_small_5d.csv`	synthetic	240	5	small balanced sanity-check dataset
`synthetic_logistic_sparse_20d.csv`	synthetic	1000	20	sparse informative coordinates, no label flips
`synthetic_logistic_noisy_20d.csv`	synthetic	1000	20	same sparse structure with 10% label flips
`synthetic_logistic_imbalanced_10d.csv`	synthetic	800	10	shifted intercept and 3% label flips
`synthetic_logistic_big_sparse_50d.csv`	synthetic	12000	50	larger sparse synthetic lasso comparison dataset
`synthetic_logistic_big_noisy_80d.csv`	synthetic	20000	80	larger noisy synthetic lasso comparison dataset

Table 3: Logistic regression datasets used by the lasso experiments.

Dataset	Samples	Dim.	Intercept	Seed	Flip prob.	Coefficient vector β
`synthetic_logistic_small_5d.csv`	240	5	0	10	0	$(2, -1.5, 1, -0.5, 0.25)$
`synthetic_logistic_sparse_20d.csv`	1000	20	0	20	0	$(2.5, -2, 1.5, -1, 0.75, 0.5, 0, \dots, 0)$
`synthetic_logistic_noisy_20d.csv`	1000	20	0	21	0.10	$(2.5, -2, 1.5, -1, 0.75, 0.5, 0, \dots, 0)$
`synthetic_logistic_imbalanced_10d.csv`	800	10	-1.2	30	0.03	$(1.5, -1.25, 1, -0.75, 0.5, 0.25, 0, \dots, 0)$
`synthetic_logistic_big_sparse_50d.csv`	12000	50	0	50	0	$(3, -2.5, 2, -1.5, 1.2, -1, 0.8, -0.6, 0.4, -0.3, 0, \dots, 0)$
`synthetic_logistic_big_noisy_80d.csv`	20000	80	-0.4	80	0.15	$(2.5, -2.2, 1.8, -1.5, 1.2, -1, 0.8, -0.6, 0.5, -0.4, 0.3, -0.2, 0, \dots, 0)$

Table 4: Synthetic logistic dataset generation parameters.

3 Experiments

This section reports the experiments conducted for the implemented subgradient and dual-averaging solvers. The benchmark experiments use convex nonsmooth test objectives with known reference optima, so their plotted objective gap is

$$f(\hat{x}_k) - f^\star,$$

where $\hat{x}_k$ is the averaged primal iterate. The lasso logistic experiments do not have a closed-form optimum. For those runs, the reference value is the train objective achieved by a matched `sklearn` SAGA run, and the plotted train-objective gap is

$$F(\hat{x}_k) - F(x_{\text{saga}}).$$

Small negative lasso train-objective gaps mean that the reported custom iterate has a lower value of the same train objective than the matched finite-iteration SAGA run. They should not be interpreted as proof that the global optimum has been reached, because the SAGA value is a numerical reference rather than an exact optimum. SDA also computes a separate primal–dual certificate gap $\delta_k(D)/S_k$, which is used for stopping. This primal–dual certificate gap is not the same quantity as the objective gaps reported in the tables and runtime plots. When D is valid, Nesterov's analysis gives

$$f(\hat{x}_k) - f_D^\star \le \frac{\delta_k(D)}{S_k}, \qquad f_D^\star = \min_{x \in F_D} f(x).$$

If $D \ge d(x^\star)$, then $f_D^\star = f^\star$, so the certificate is an upper bound on the true objective gap. The converse is not true: a small objective gap does not imply a small certificate gap. The certificate is based on the averaged affine lower model accumulated from all past subgradients, so it measures a primal–dual model gap rather than only the objective value at $\hat{x}_k$. This averaged model can remain loose even when the averaged primal point is already close to optimal, making the certificate-based stopping rule harder to satisfy than a direct objective-gap target. The report focuses on objective gaps because they measure the actual optimization error when a reference optimum is available and because they are comparable across SDA, projected subgradient, and SAGA. The certificate gap remains important as an SDA stopping rule, but it is method-specific, can be conservative, and can be misleading when the radius condition fails. For this reason the experiments use a fixed 1000-iteration budget when comparing methods and treat certificate-based early stopping as an additional SDA diagnostic rather than as the only measure of progress.

The experimental design follows the distinction made in Nesterov's primal–dual subgradient framework: the method maintains averaged primal iterates and a dual aggregate, and the weighted variant changes the scale used to aggregate subgradients [1]. The conclusions below should therefore be read against the same theoretical background: the guarantees are worst-case $O(1/\sqrt{k})$ subgradient guarantees that depend on a valid prox radius and uniformly bounded subgradients, not predictions that one variant dominates every finite benchmark. The geometry experiments are also consistent with the mirror-descent view that the prox should encode useful problem geometry rather than an arbitrary norm change [4]. The stochastic lasso comparison is related to the regularized stochastic dual averaging literature, where averaging sampled gradients is natural for finite-sum learning objectives [2]. The SAGA solver is used only as an empirical reference for lasso logistic regression because SAGA is designed for finite-sum composite objectives with nonsmooth regularizers [3]; lasso logistic regression is a standard sparse classification benchmark class [5].

3.1 Nonsmooth Objective Grid

Purpose and setup. The first experiment tests Euclidean primal–dual subgradient methods on a broad grid of nonsmooth convex objectives. The objective registry contains nine problems: absolute-value, max-affine, shifted ℓ_∞, weighted ℓ_1, and ill-conditioned variants. Each problem has a known reference optimum. The grid compares:

- simple Euclidean SDA,
- weighted Euclidean SDA,
- projected subgradient descent.

For SDA, the grid uses prox-radius parameters $D \in \{0.5, 2, 8, 32\}$ and multiplier values $\gamma_{\text{mult}} \in \{0.1, 0.25, 0.5, 1, 2, 4\}$. Here γ_{mult} multiplies the baseline SDA step scale γ used by the implementation. Both unrestricted and restricted-to-F_D runs are included. Projected subgradient descent uses the same D values and $\alpha \in \{0.1, 0.25, 0.5, 1, 2\}$. The experiment contains 1224 runs.

Method	Runs	Median final objective gap	Best objective gap	Median runtime (s)
Projected subgradient	360	$8.35 \cdot 10^{-2}$	$7.45 \cdot 10^{-4}$	$4.51 \cdot 10^{-3}$
SDA simple	432	$2.77 \cdot 10^{-1}$	$3.72 \cdot 10^{-4}$	$9.35 \cdot 10^{-3}$
SDA weighted	432	$1.75 \cdot 10^{-1}$	$3.35 \cdot 10^{-4}$	$1.02 \cdot 10^{-2}$

Table 5: Aggregate final objective gaps $f(\hat{x}) - f^\star$ for the nonsmooth objective grid.

Unrestricted SDA and the radius condition. Figure 1 uses the default SDA step choice $\gamma_{\text{mult}} = 1$, simple dual averaging, and several values of D. Its purpose is to show a failure mode of the primal–dual certificate gap used for stopping when the radius is chosen too small. In the primal–dual certificate proof, D is required to satisfy

$$D \geq d(x^\star), \qquad \text{equivalently} \qquad x^\star \in F_D,$$

where $F_D = \{x : d(x) \leq D\}$ is the prox ball and $x^\star$ is a minimizer of the original problem. This is not a cosmetic assumption. If $x^\star \in F_D$, then the subgradient inequality gives

$$f(x_i) - f(x^\star) \leq \langle g_i, x_i - x^\star \rangle.$$

Because $x^\star$ is then an admissible point in the maximization defining the primal–dual certificate gap, we also have

$$\sum_i \lambda_i \langle g_i, x_i - x^\star \rangle \leq \max_{x \in F_D} \sum_i \lambda_i \langle g_i, x_i - x \rangle = \delta_k(D).$$

Combining this with convexity of f yields the primal–dual certificate gap

$$f(\hat{x}_k) - f(x^\star) \leq \frac{\delta_k(D)}{S_k}.$$

This is the mathematical reason SDA can safely stop when

$$\frac{\delta_k(D)}{S_k} \leq \varepsilon$$

provided that $D \geq d(x^\star)$.

If $D < d(x^\star)$, the proof breaks exactly at the step where $x^\star$ is inserted into the maximization over F_D. The computed $\delta_k(D)$ is then no longer a valid primal–dual certificate gap for the original unconstrained problem. It can certify, at best, behavior relative to the restricted problem

$$\min_{x \in F_D} f(x).$$

When the iterates are not themselves restricted to F_D, even this restricted-problem interpretation must be read carefully: $\hat{x}_k$ may lie outside F_D, so the value is not a feasible restricted-solution certificate. In this regime a negative computed primal–dual certificate gap is not paradoxical: the nonnegativity and upper-bound interpretation of $\delta_k(D)$ rely on the same radius condition. Therefore a negative value $\delta_k(D)/S_k < \varepsilon$ may trigger early stopping even while the true objective gap $f(\hat{x}_k) - f(x^\star)$ is still large. This is the main phenomenon illustrated by Figure 1. In the stored nonsmooth-grid results, 93 of the 864 SDA runs ended with a negative final certificate gap while still having true objective gap larger than 10^{-3}. All 93 cases were unrestricted runs with $D \in \{0.5, 2\}$. This supports the mathematical interpretation: the failure is associated with using the certificate outside its radius assumption, not with the objective-gap metric itself.

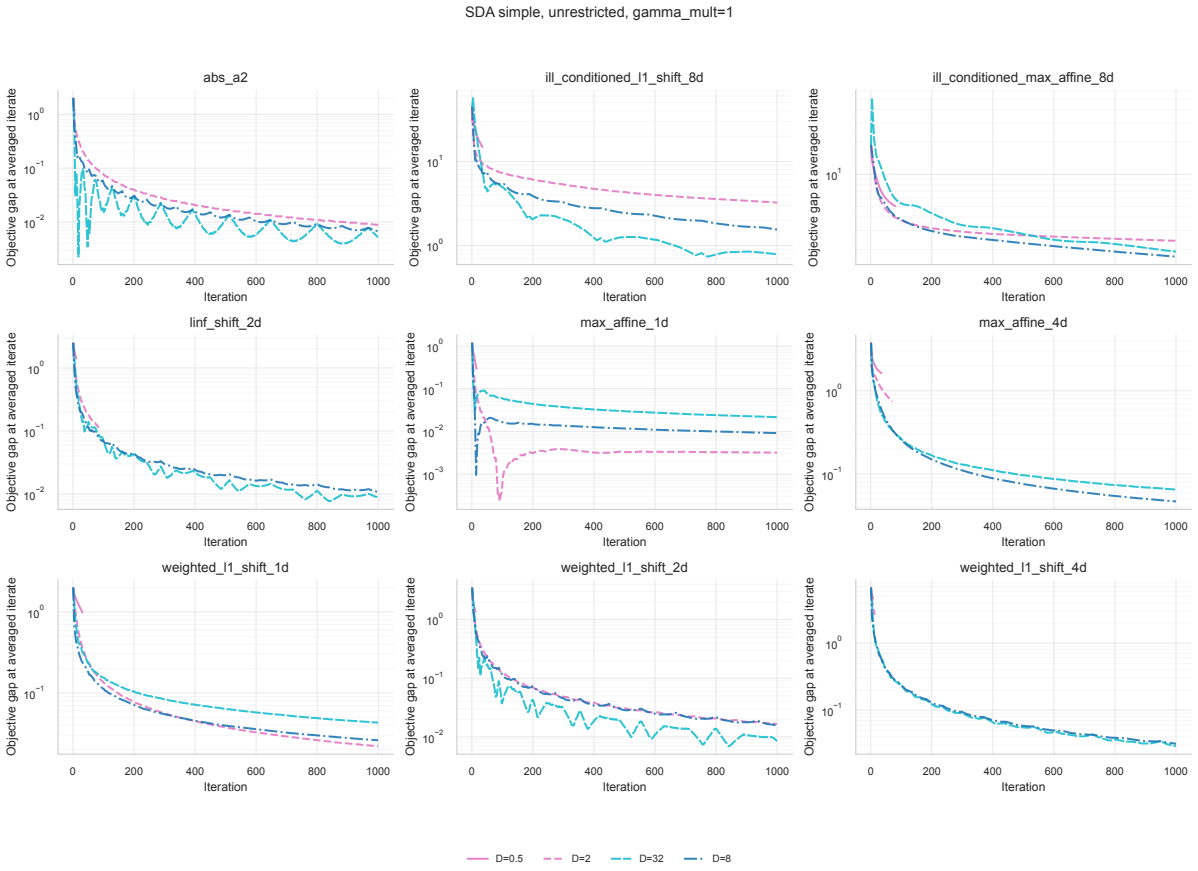

Figure 1: Default unrestricted simple Euclidean SDA trajectories. The plot illustrates that an invalid radius choice can produce a small or negative primal–dual certificate gap while the true objective gap remains large.

Restricted SDA. Figure 2 repeats the default SDA comparison but forces the iterates to remain inside F_D. In this case the algorithm is solving the restricted problem on the prox ball. The primal–dual certificate gap is then consistent with the feasible problem being solved, and the fake early convergence caused by an invalid primal–dual certificate gap is removed.

The price is that the optimization problem has changed: the reported solution is a solution for $\min_{x \in F_D} f(x)$, not necessarily for the original problem over the larger ambient set Q. Thus restriction is useful for illustrating the radius assumption, but it is less general than applying SDA to the original problem with prior knowledge that $x^\star \in F_D$.

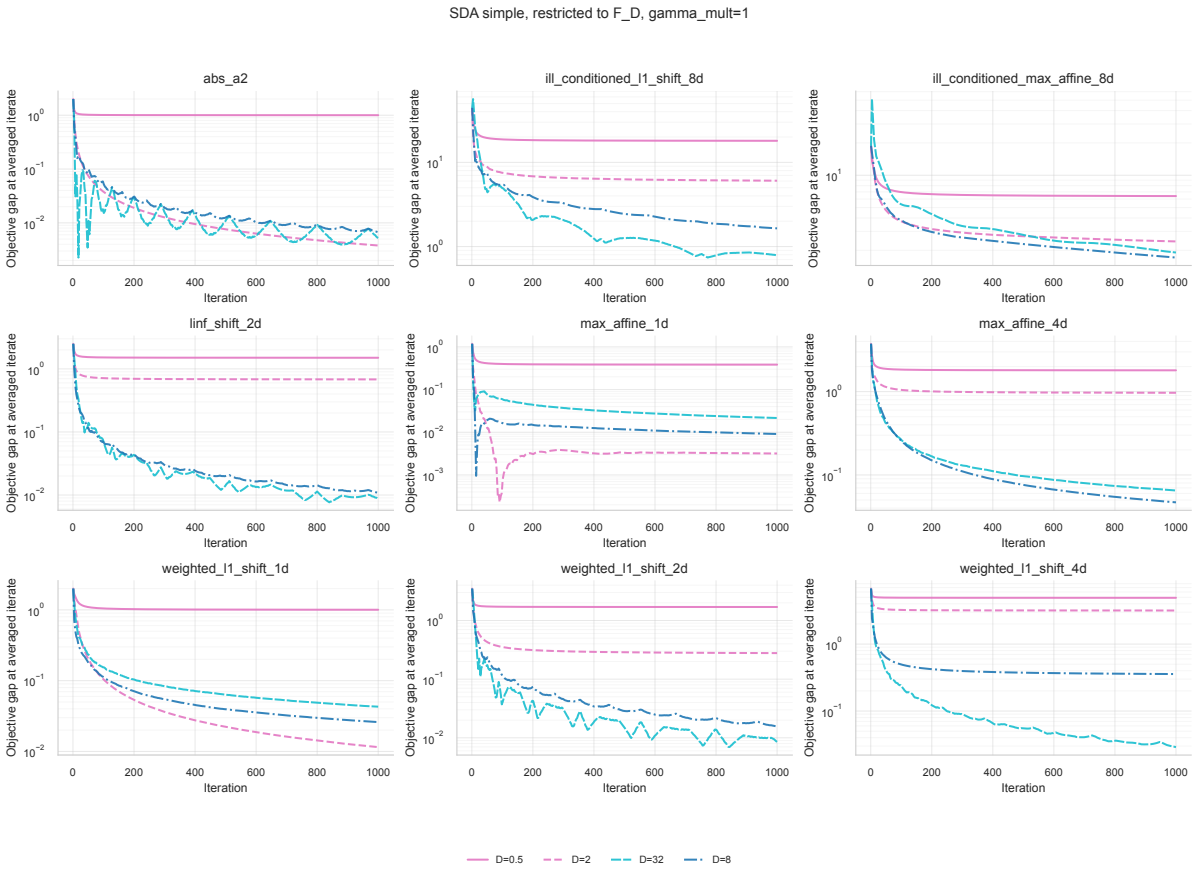

Figure 2: Default restricted simple Euclidean SDA trajectories. Restricting to F_D prevents invalid primal–dual-certificate-based stopping, but changes the optimization problem to the prox-ball-restricted problem.

Figure 3 compares unrestricted and restricted runs more directly. Even when D is large enough to contain a minimizer, the two variants may follow different trajectories. The unrestricted method can temporarily move outside F_D and later return, whereas the restricted method forbids such movement. Therefore the restricted variant is not simply a safer implementation of the same trajectory; it is a different constrained algorithm. For the remaining benchmark discussion, the unrestricted runs are the main focus because they represent the intended use of SDA for the original problem. This interpretation requires external knowledge, or a conservative bound, that the selected D contains a minimizer.

Step-size and radius sensitivity. Figure 4 fixes $D = 8$ and compares several γ_{mult} values. The implemented step is a multiplier of the Nesterov baseline step, so $\gamma_{\text{mult}} = 1$ corresponds to the proposed theoretical scale used by the code. The parameter γ controls how strongly the accumulated dual subgradient model affects the next prox-center minimization: larger values increase the prox regularization and therefore produce smaller primal moves in response to the observed subgradients, while smaller values weaken the regularization and produce larger moves. The figure shows that the baseline multiplier is not always the best empirical choice in terms of objective-gap decrease. This is mathematically reasonable: the theoretical step is based on

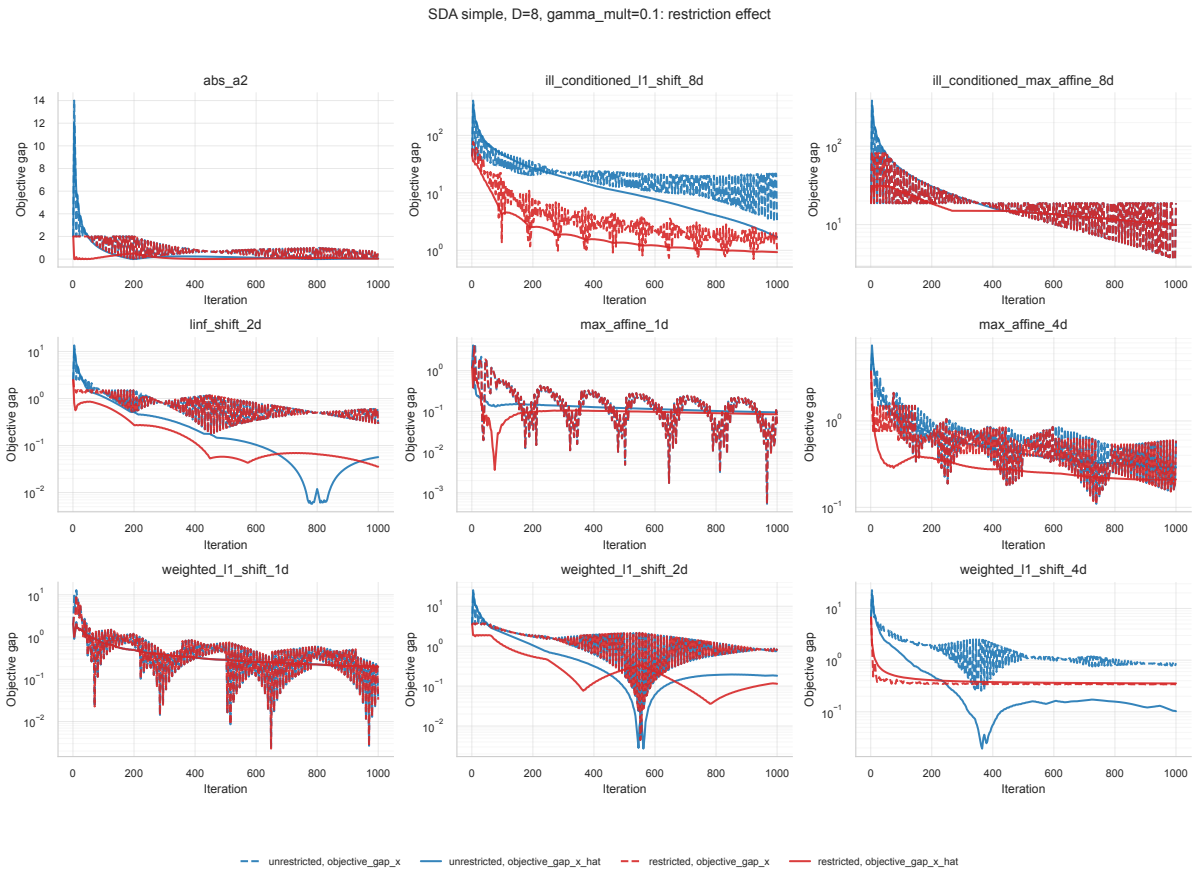

Figure 3: Current-iterate and averaged-iterate objective gaps for restricted and unrestricted SDA. The trajectories can differ even when the minimizer lies inside the chosen prox radius.

worst-case bounds, while the benchmark objectives have different local geometry, subgradient magnitudes, and distances from the initial point to the minimizer. The method conclusion is that γ_{mult} should be treated as a tuning parameter around the theoretically balanced scale, not as a universally optimal constant.

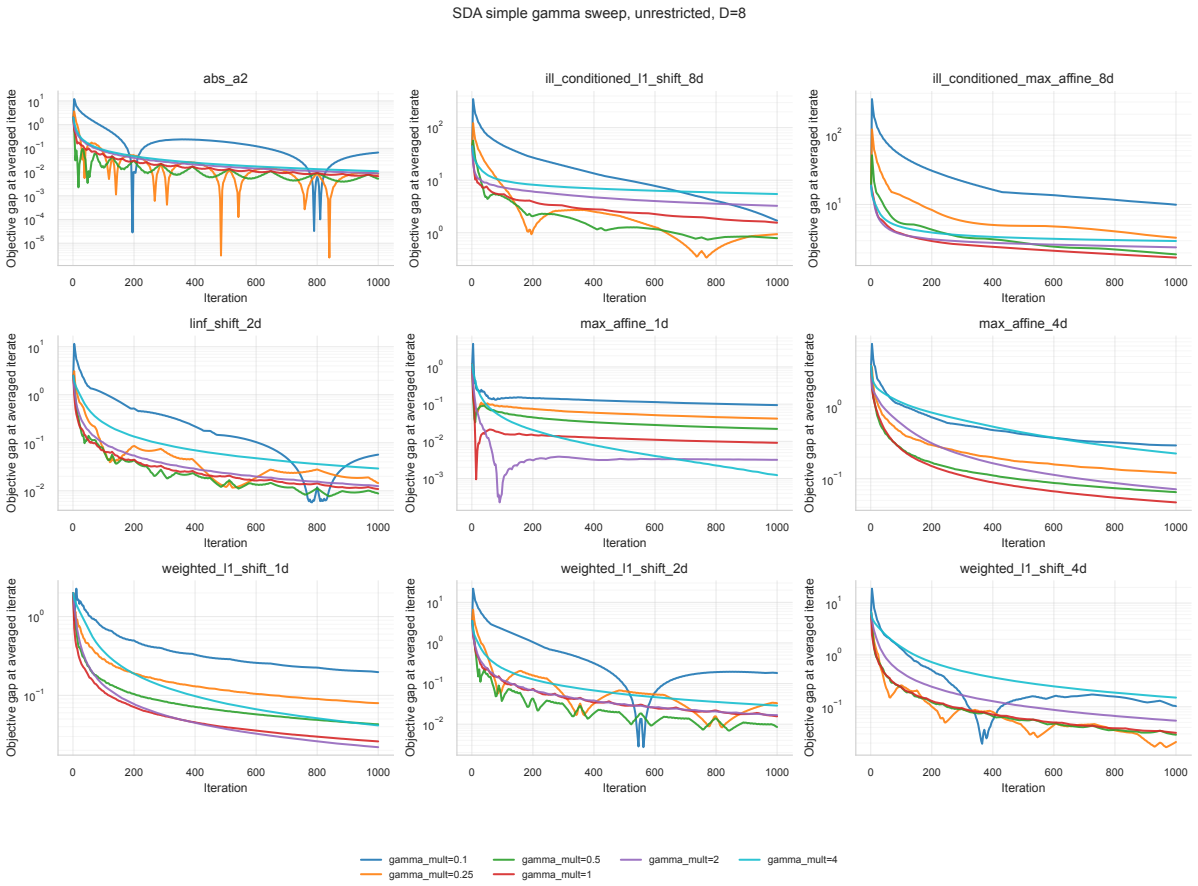

Figure 4: Sensitivity of nonsmooth benchmark performance to γ_{mult} at fixed $D = 8$. The baseline $\gamma_{\text{mult}} = 1$ is not uniformly optimal.

Figure 5 shows that γ and D should be considered jointly. The radius D determines the prox ball used in the primal–dual certificate gap and changes the scale of the theoretical step through the diameter/radius bound. The step size determines how quickly the dual aggregate is translated into primal movement. If D is too small, the primal–dual certificate gap may not apply to the original problem. If D is very large, the corresponding baseline scale can become too weak because $\gamma^\star = L/\sqrt{2D}$ decreases with D, unless the multiplier is adjusted. Thus a multiplier that is effective for one radius may be poor for another radius. In practice, D should be chosen from prior knowledge or a conservative bound on the solution radius, and γ_{mult} should be tuned relative to that choice.

Method comparison. Figures 6 and 7 compare simple SDA, weighted SDA, and projected subgradient descent. On these benchmark functions, the standard projected subgradient method can behave as well as, or better than, SDA for several objectives, and it is usually faster in wall-clock time. Table 5 reflects this: projected subgradient has the smallest median final objective gap over the full nonsmooth grid, while the best single run is achieved by weighted SDA. This should not be interpreted as a contradiction of SDA theory. The methods solve slightly different algorithmic subproblems, the grid includes invalid small-radius SDA runs, and the theoretical guarantees are worst-case bounds rather than predictions of dominance on every

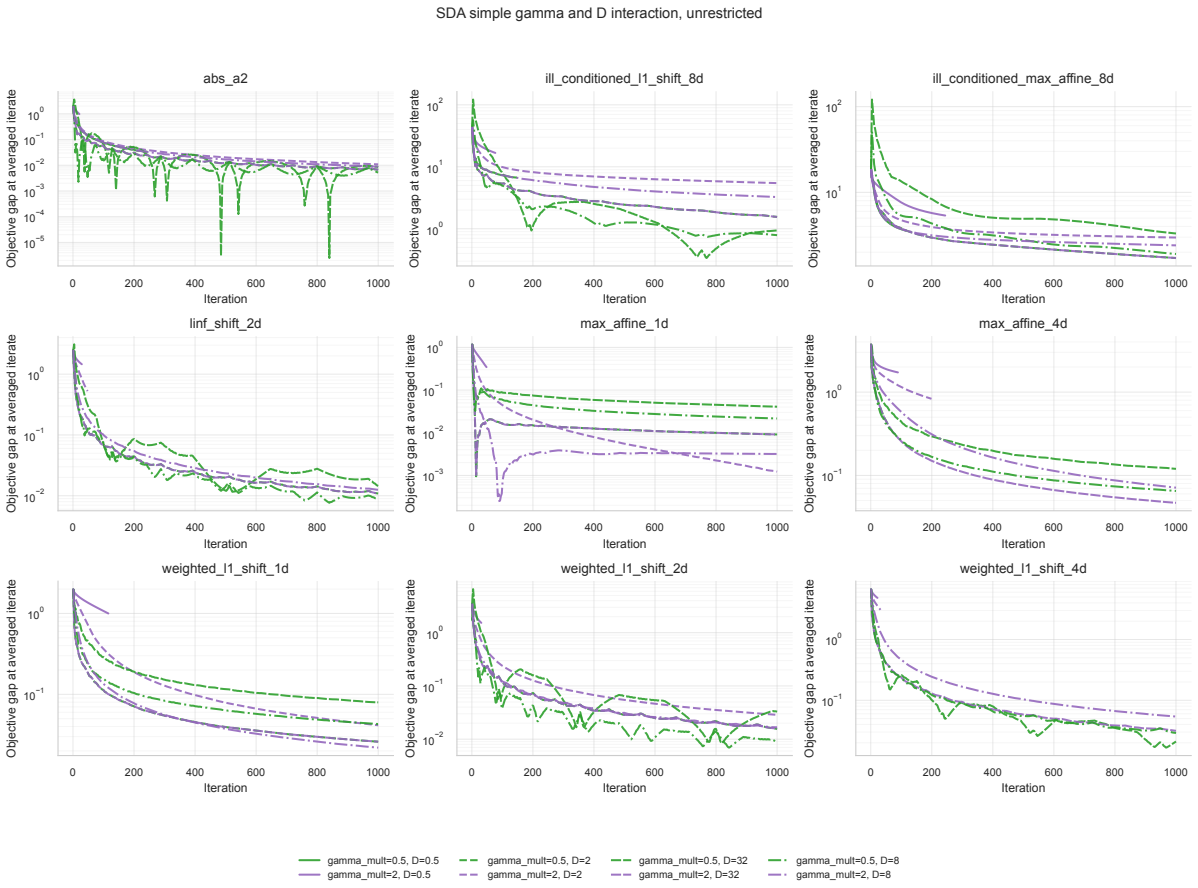

Figure 5: Interaction between prox-radius parameter D and step multiplier γ_{mult}. The useful parameter is the pair $(D, \gamma_{\text{mult}})$, not either value in isolation.

small benchmark. The method-level conclusion from Figure 6 is therefore comparative rather than absolute: projected subgradient is a strong baseline on these low-dimensional benchmarks, while weighted SDA can be best after tuning on some individual objectives.

The runtime plot also clarifies why final objective gaps must be read together with the radius condition. Some SDA points with very small runtime and large true objective gap correspond to runs that stopped early because D was too small and the computed primal–dual certificate gap became invalid or negative. These are not successful fast convergences. They are examples of the fake convergence mechanism shown in Figure 1. Consequently, Figure 7 should be read as a runtime–accuracy plot with certificate validity checked separately.

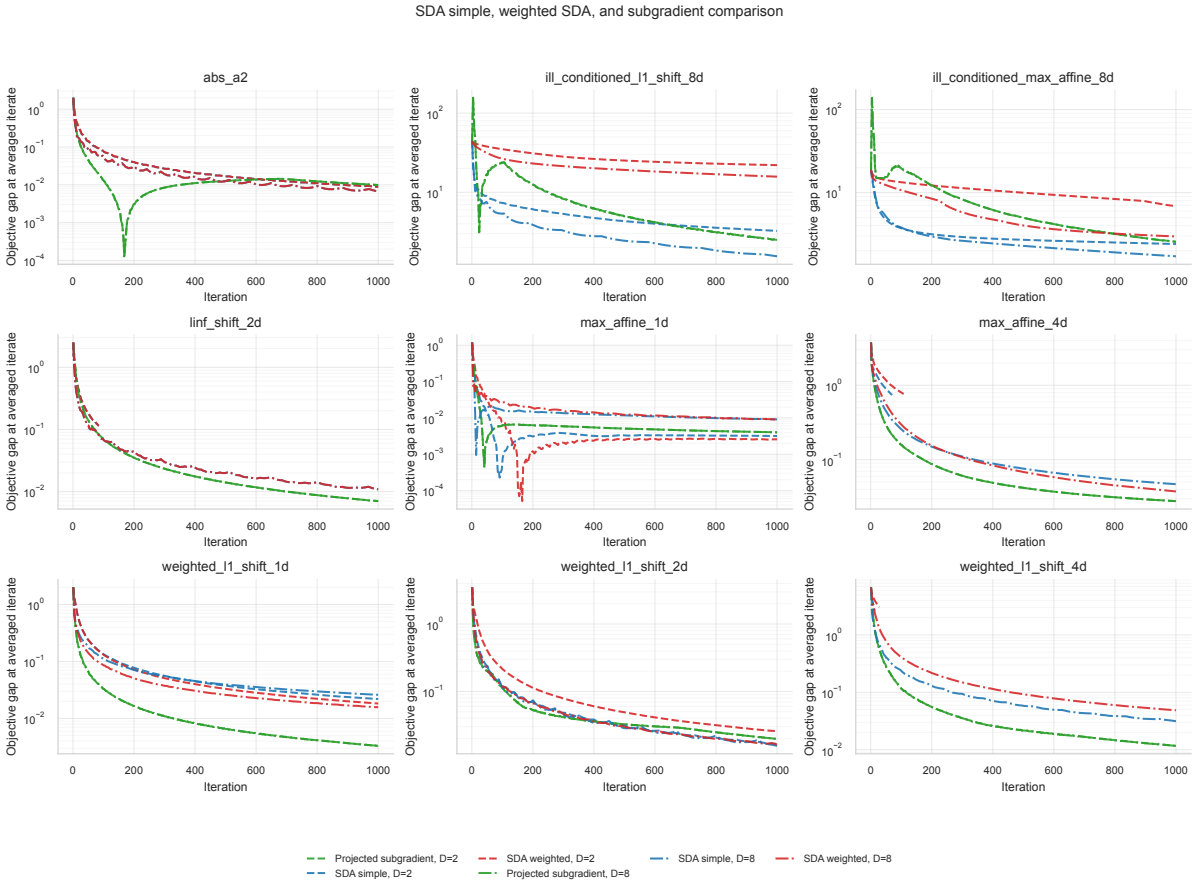

Figure 6: Default comparison of simple SDA, weighted SDA, and projected subgradient descent on nonsmooth benchmark objectives.

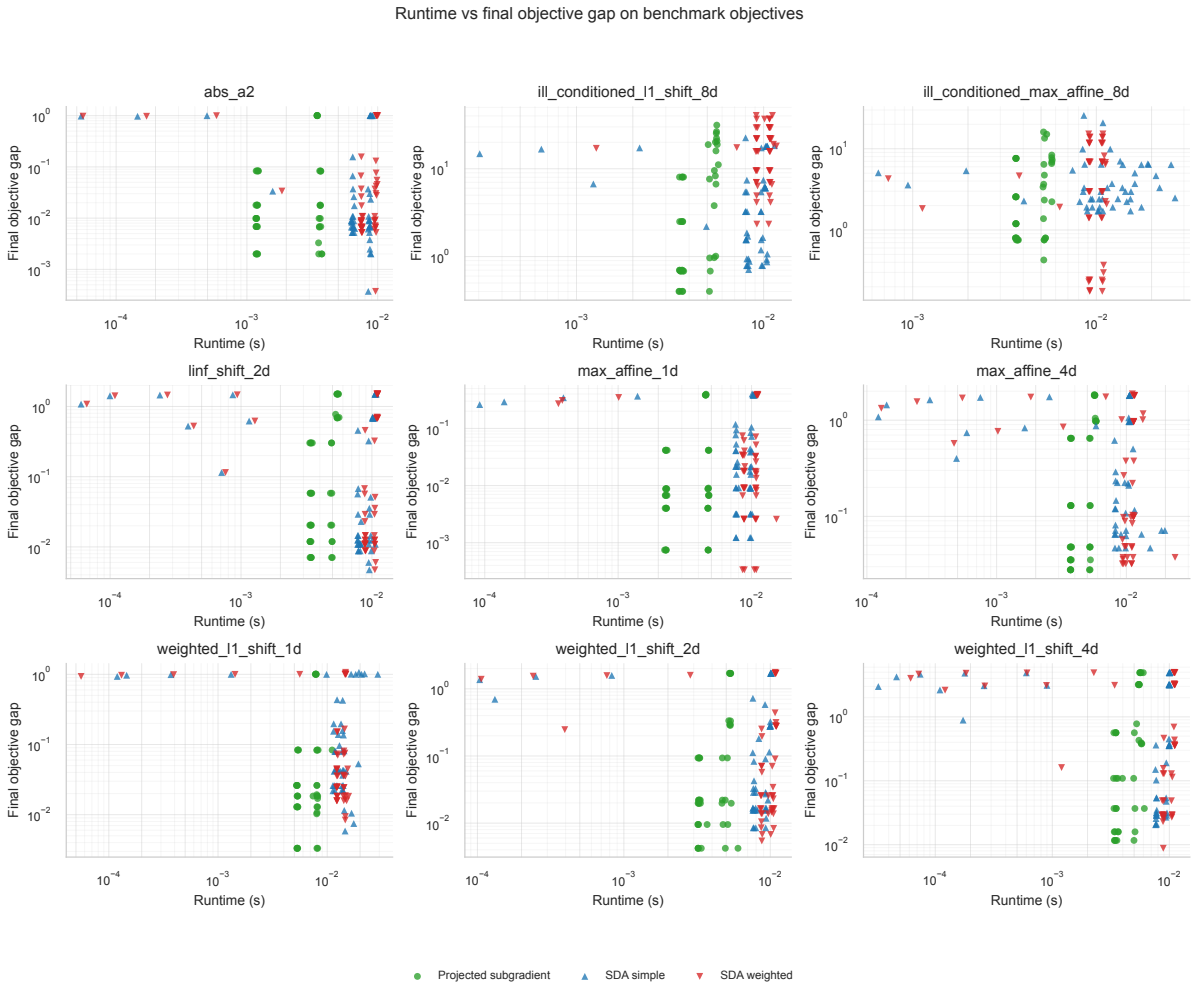

Figure 7: Runtime versus final true objective gap for nonsmooth benchmark objectives. Low-runtime, high-objective-gap SDA points are often invalid early stops caused by too small a radius.

3.2 Prox Geometry Comparison

Purpose and setup. The second experiment isolates the effect of the prox geometry. It uses three coordinate-imbalanced nonsmooth objectives: an ill-conditioned shifted ℓ_1 objective, an ill-conditioned max-affine objective, and a weighted ℓ_1 objective. The methods are Euclidean SDA and weighted Euclidean SDA, each with simple and weighted dual averaging. For the weighted Euclidean prox, three diagonal profiles are tested: uniform, coordinate-matched, and inverse-coordinate-matched. The grid uses $D \in \{2, 8, 32\}$, $\gamma_{\text{mult}} \in \{0.5, 1, 2\}$, and both restricted and unrestricted runs, for 432 runs total.

Method	Runs	Median final objective gap	Best objective gap	Median runtime (s)
SDA simple, Euclidean prox	54	1.36	$1.24 \cdot 10^{-2}$	$1.45 \cdot 10^{-2}$
SDA simple, weighted prox	162	2.18	$1.24 \cdot 10^{-2}$	$1.60 \cdot 10^{-2}$
SDA weighted, Euclidean prox	54	3.14	$1.94 \cdot 10^{-2}$	$1.51 \cdot 10^{-2}$
SDA weighted, weighted prox	162	2.92	$1.94 \cdot 10^{-2}$	$1.89 \cdot 10^{-2}$

Table 6: Aggregate final objective gaps $f(\hat{x}) - f^\star$ in the prox-geometry experiment.

Figure 8 shows that diagonal geometry is not uniformly beneficial. The coordinate-matched profile can improve behavior on objectives whose anisotropy agrees with that profile, but the inverse profile is a useful negative control: it emphasizes the wrong coordinates and may slow convergence. This is consistent with mirror-descent intuition. A prox function improves a method only when it represents the geometry of the feasible set and the subgradients well; an arbitrary change of norm is not a free acceleration. The selected $D = 32$, $\gamma_{\text{mult}} = 1$, unrestricted runs show this directly: the coordinate-scaled simple weighted prox improves the ill-conditioned ℓ_1 shifted objective relative to the uniform Euclidean prox, but the inverse-scaled profile is much worse on the same objective. The method conclusion is that weighted geometry should be justified by coordinate scale information, not added automatically.

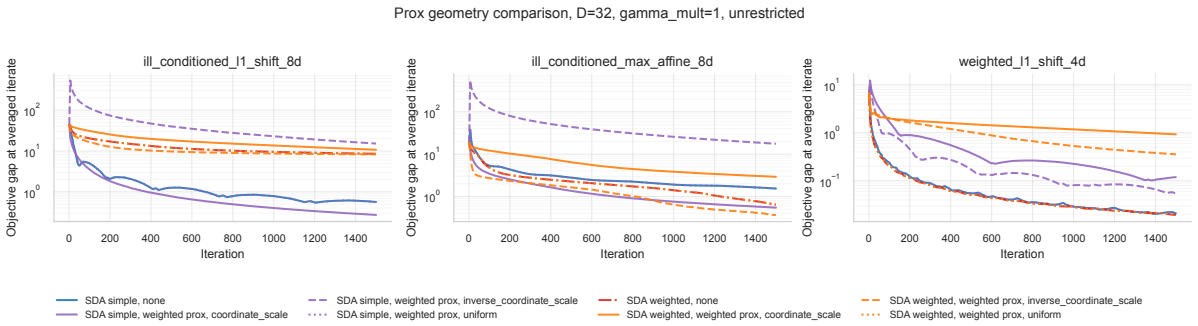

Figure 8: Euclidean and weighted Euclidean prox choices on coordinate-imbalanced objectives.

The runtime summary in Figure 9 leads to the same conclusion. Weighted geometry introduces a small additional computational cost in these low-dimensional tests, and its optimization benefit depends on the objective. The median objective gaps in Table 6 are therefore best read together with the per-objective panels rather than as an absolute ranking of all geometries. Thus Figure 9 does not support a runtime-only preference for the weighted prox in this implementation; its value is conditional on reducing enough iterations or objective gap to offset the extra per-iteration work.

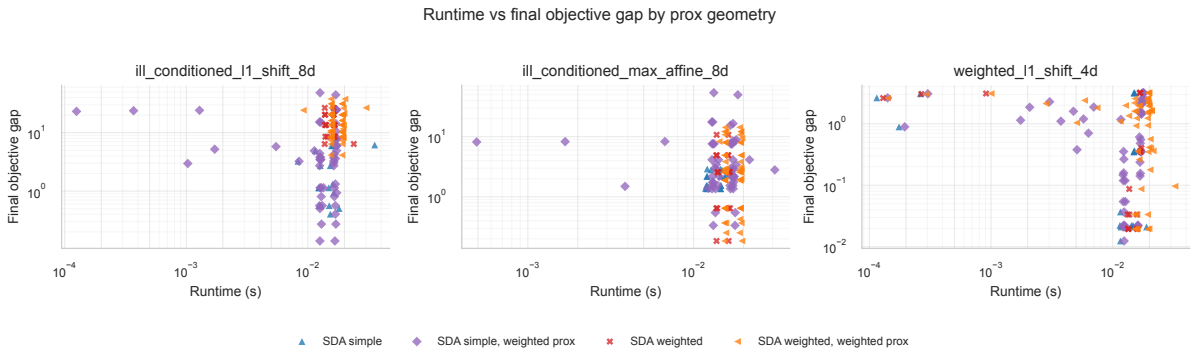

Figure 9: Runtime versus final true objective gap for prox-geometry runs.

3.3 Entropy Geometry on Simplex Objectives

Purpose and setup. The third experiment tests entropy prox on simplex-constrained objectives. The simplex is a natural setting for entropy geometry because the primal geometry is ℓ_1-like and the dual norm is ℓ_∞. The objectives are `simplex_linear_8d`, `simplex_max_affine_8d`, and `simplex_sparse_max_affine_16d`. Simple and weighted entropy SDA are run with $D \in \{1, 2, 3\}$ and $\gamma_{\text{mult}} \in \{0.1, 0.5, 1, 4\}$, for 72 runs total.

Method	Runs	Median final objective gap	Best objective gap	Median runtime (s)
Entropy SDA simple	36	$8.37 \cdot 10^{-3}$	$1.09 \cdot 10^{-3}$	$6.16 \cdot 10^{-1}$
Entropy SDA weighted	36	$8.13 \cdot 10^{-3}$	$1.09 \cdot 10^{-3}$	$6.57 \cdot 10^{-1}$

Table 7: Aggregate final objective gaps $f(\hat{x}) - f^\star$ for entropy prox on simplex objectives.

Figure 10 shows that the simple and weighted entropy variants are close in median final objective gap. The best run on `simplex_linear_8d` is achieved by simple entropy SDA with $D = 3$ and $\gamma_{\text{mult}} = 0.1$, while the sparse max-affine objective favors the weighted entropy variant in the best run. This supports a moderate conclusion: entropy is appropriate for the simplex constraint, but the weighted averaging rule is not uniformly dominant on these three objectives. The figure also reinforces the parameter-sensitivity conclusion from the Euclidean grid: changing D and γ_{mult} can matter as much as changing from simple to weighted averaging.

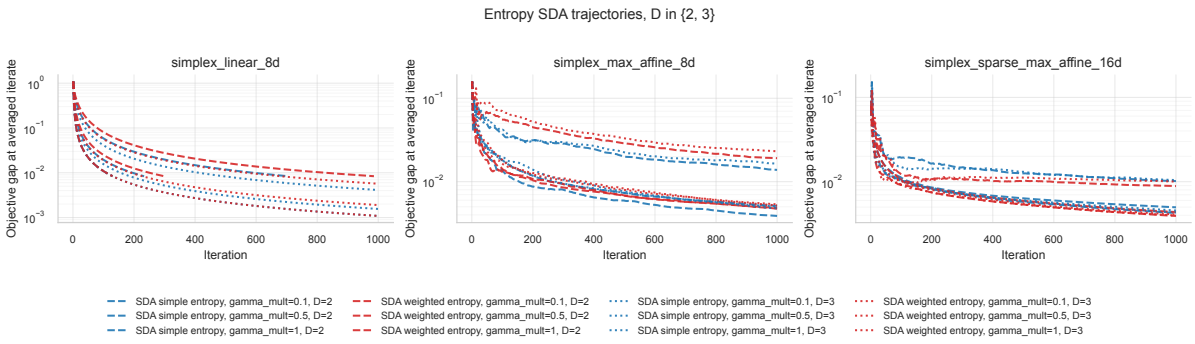

Figure 10: Simple and weighted entropy SDA on simplex-constrained objectives.

Figure 11 shows that the entropy runs are slower than the small Euclidean benchmark runs, with median runtimes around 0.6 seconds. That runtime reflects the current implementation and the denser simplex projection/prox computations; it should not be interpreted as a lower bound on entropy mirror descent. The method conclusion from this plot is that entropy prox is

geometrically suitable for simplex constraints, but its practical value must be judged together with implementation cost.

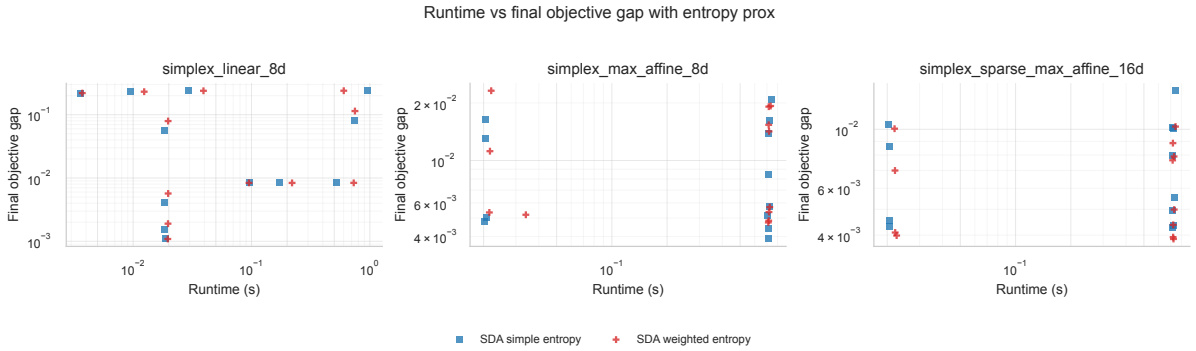

Figure 11: Runtime versus final true objective gap for entropy-prox simplex runs.

3.4 Lasso Logistic Sparsity

Purpose and setup. The fourth experiment evaluates the methods on binary lasso logistic regression datasets. The objective has the form

$$F(w,b) = \frac{1}{n}\sum_{i=1}^{n} \log\bigl(1+\exp(x_i^\top w + b)\bigr) - y_i(x_i^\top w + b) + \frac{\lambda}{n}\|w\|_1,$$

where the intercept is not penalized. The current grid uses $\lambda = 1$, unrestricted iterates, seed 0, and a train/test split of 0.8/0.2. The datasets are `iris`, four synthetic logistic datasets, `breast_cancer_wdbc`, `banknote_authentication`, and `spambase`. The custom methods are simple SDA, weighted SDA, and projected subgradient; sklearn SAGA is included as the empirical reference.

Method	Runs	Median train-objective gap to SAGA	Best train-objective gap	Median runtime (s)	Median accuracy
Projected subgradient	72	$3.38 \cdot 10^{-2}$	$4.02 \cdot 10^{-4}$	$3.60 \cdot 10^{-2}$	0.878
SDA simple	96	$2.03 \cdot 10^{-2}$	$6.26 \cdot 10^{-5}$	$4.29 \cdot 10^{-2}$	0.878
SDA weighted	96	$1.74 \cdot 10^{-5}$	$-1.43 \cdot 10^{-3}$	$3.87 \cdot 10^{-2}$	0.888
sklearn SAGA	8	0	0	$3.75 \cdot 10^{-3}$	0.888

Table 8: Lasso logistic final train-objective gaps to the matched sklearn SAGA reference.

Figure 12 compares the methods over D. This figure fixes $\gamma_{\text{mult}} = 1$ for SDA and $\alpha = 1$ for projected subgradient. Weighted SDA is the most reliable method in this lasso grid: it has the smallest median train-objective gap to the SAGA reference and is the best custom method on every dataset in the final-run CSV. This is a statement about this parameter grid, not a general theorem. The lasso objective is convex but nonsmooth because of the ℓ_1 term, and the experiment uses subgradients of the penalty rather than the exact proximal regularized dual averaging update studied by Xiao [2]. In the stored final-run CSV, the best custom run for each of the 8 lasso datasets uses weighted SDA with $D = 8$; the best γ_{mult} varies across datasets from 0.25 to 2. Thus Figure 12 supports weighted SDA in this grid, but it also shows that the radius and step scale remain empirical choices. Table 9 lists the Figure 12 runs where the final train objective is below the matched SAGA reference.

Figure 13 fixes $D = 32$ and varies the SDA multiplier γ_{mult} or projected-subgradient step α. The parameter sensitivity is substantial: a method that is competitive at one step scale can be much worse at another. The absolute train-objective view in Figure 14 confirms that the smallest train-objective gaps correspond to curves approaching the matched SAGA train

Table 9: Negative final lasso train-objective gaps in the D-sweep setting of Figure 12

Dataset	Method	D	Train objective	SAGA train objective	Gap to SAGA	Test accuracy
`breast_cancer_wdbc.csv`	Weighted SDA	128	$6.7885 \cdot 10^{-2}$	$6.8065 \cdot 10^{-2}$	$-1.80 \cdot 10^{-4}$	0.947
`spambase.csv`	Weighted SDA	32	$2.1506 \cdot 10^{-1}$	$2.1585 \cdot 10^{-1}$	$-7.94 \cdot 10^{-4}$	0.923
`spambase.csv`	Weighted SDA	128	$2.1500 \cdot 10^{-1}$	$2.1585 \cdot 10^{-1}$	$-8.47 \cdot 10^{-4}$	0.923

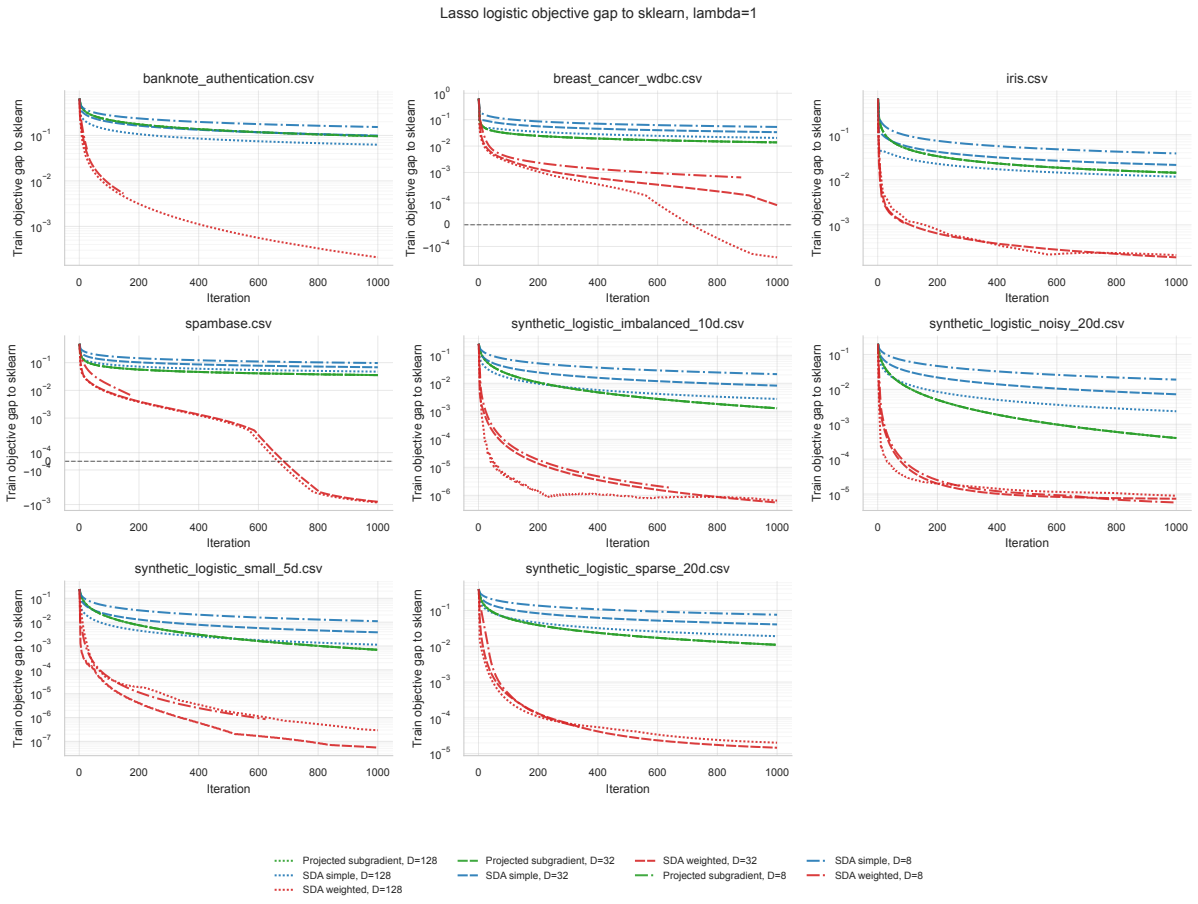

Figure 12: Effect of D on lasso logistic train-objective gap to the sklearn SAGA reference, with $\gamma_{\text{mult}} = 1$ for SDA and $\alpha = 1$ for projected subgradient.

objective, rather than to an artifact of plotting only differences. The gap plot uses a signed logarithmic scale with a zero line, so negative segments indicate train objectives below the matched SAGA reference.

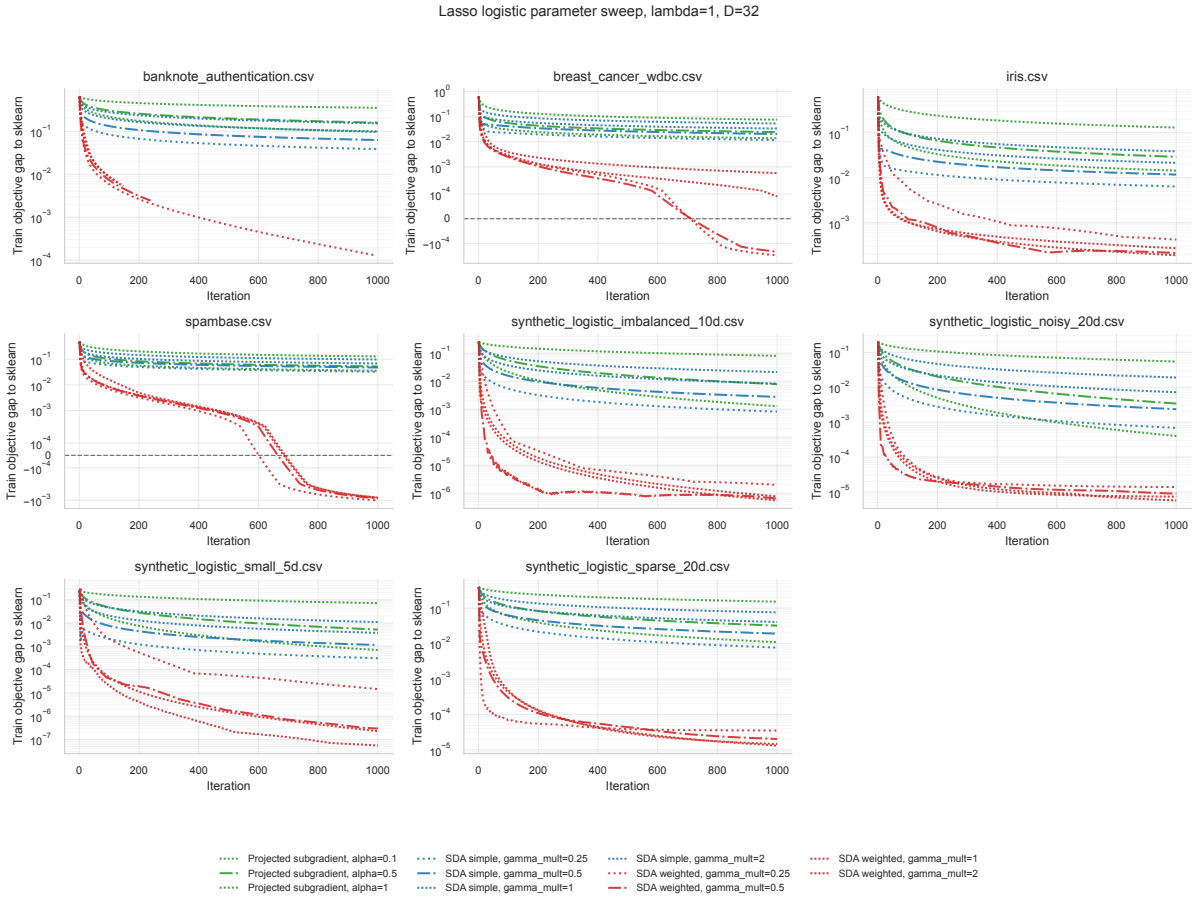

Figure 13: Step-parameter sweep for lasso logistic train-objective gaps at $D = 32$, shown on a signed logarithmic scale with zero marked by a dashed line.

Figure 15 compares iterate norms against the SAGA solution norm. A decreasing train-objective gap does not require the iterate norm to match the SAGA norm exactly, especially when the lasso problem has correlated features or multiple nearly equivalent sparse predictors. The norm plot is therefore used as a diagnostic for scale and regularization behavior, not as a separate optimality certificate.

The runtime plot in Figure 16 shows that sklearn SAGA is faster on these small and medium lasso datasets, which is expected for a highly optimized finite-sum solver. Among the custom solvers, weighted SDA reaches near-reference train objective values at runtimes comparable to the other custom methods. Since the SAGA reference is itself numerical, the runtime plot should be interpreted as an empirical comparison to a strong implementation baseline, not as a certificate of global optimality. The method conclusion is that the custom weighted SDA implementation is competitive in train objective within the custom solver family, while SAGA remains the appropriate optimized baseline for this finite-sum composite problem.

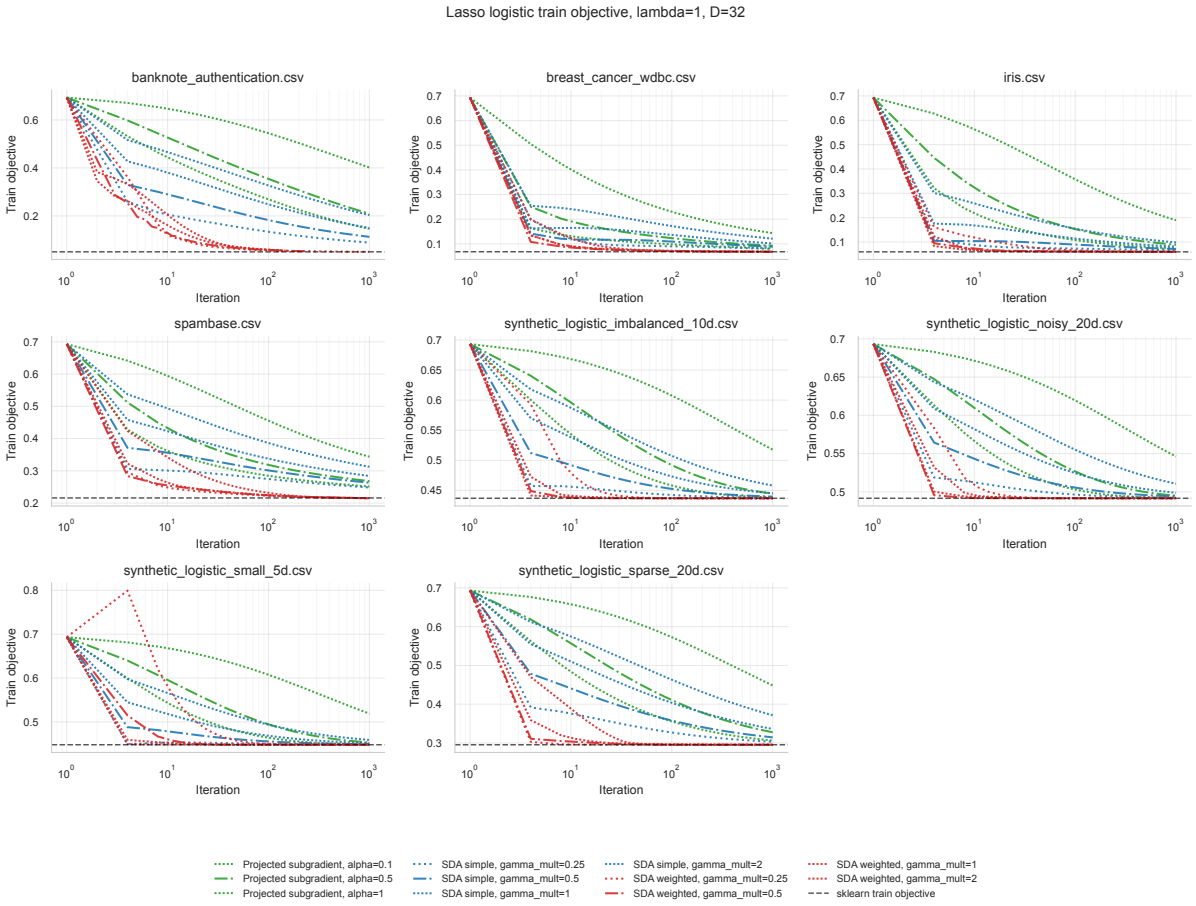

Figure 14: Absolute train objectives for the lasso parameter sweep, with the sklearn SAGA reference.

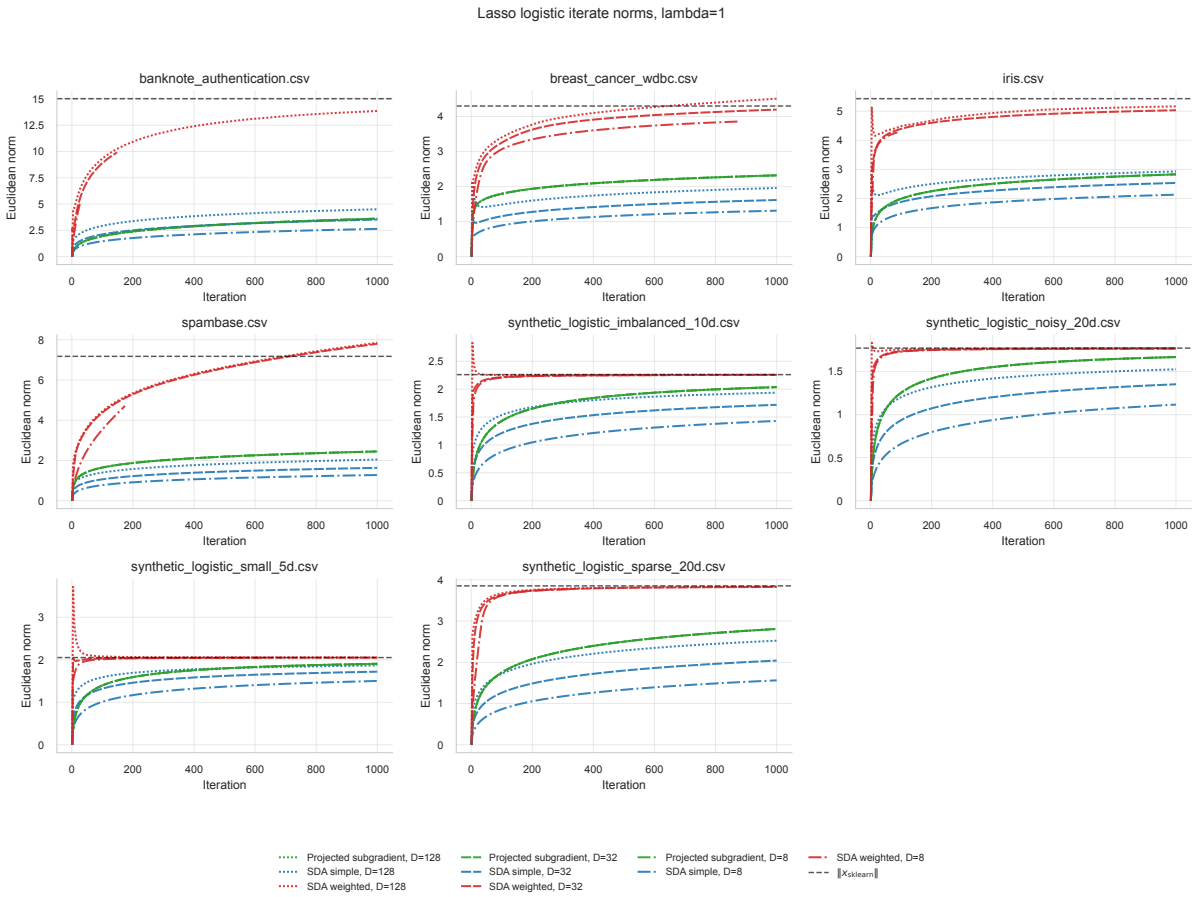

Figure 15: Norms of lasso averaged iterates compared with the sklearn SAGA solution norm.

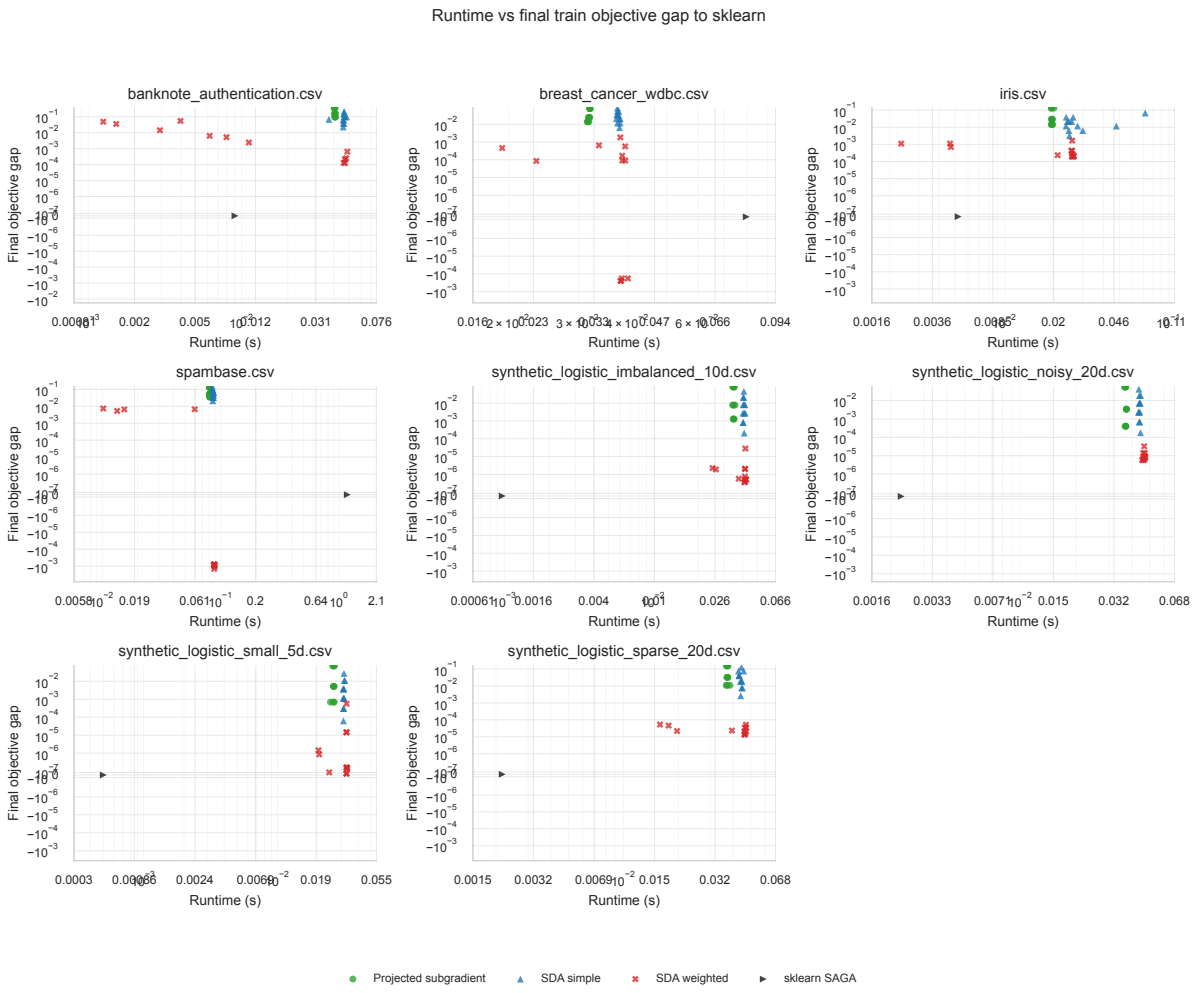

Figure 16: Runtime versus final train-objective gap to sklearn SAGA for lasso logistic datasets.

3.5 Stochastic Simple Averages versus Weighted SDA on Synthetic Lasso

Purpose and setup. The fifth experiment compares weighted full-gradient SDA with stochastic simple averages (SSA) on synthetic lasso logistic datasets only. The synthetic subset contains the four original synthetic datasets and two larger generated datasets: a sparse 50-dimensional problem with 12000 samples and a noisier 80-dimensional problem with 20000 samples. The comparison fixes $D = 128$, uses $\lambda = 1$, and sweeps γ_{mult} values $0.25, 0.5, 1, 2$. SSA samples one training example per iteration and uses the resulting stochastic subgradient in the simple dual-average update. The sampled subgradient is an unbiased estimator of the empirical logistic loss gradient component, with the deterministic lasso subgradient added using the same λ/n scaling as the full objective.

Method	Runs	Median train-objective gap to SAGA	Best train-objective gap	Median runtime (s)	Median accuracy
SDA weighted	24	$6.95 \cdot 10^{-6}$	$5.40 \cdot 10^{-8}$	$4.57 \cdot 10^{-2}$	0.840
SSA	24	$4.67 \cdot 10^{-2}$	$1.53 \cdot 10^{-3}$	$2.64 \cdot 10^{-2}$	0.828
sklearn SAGA	6	0	0	$2.40 \cdot 10^{-3}$	0.840

Table 10: Synthetic-only SSA comparison. Train-objective gaps are measured against matched sklearn SAGA.

The runtime summary in Figure 17 shows the main trade-off. SSA is cheaper per iteration and has a lower median runtime than weighted SDA in this grid, but its median final train-objective gap is much larger. Weighted SDA is slower because it evaluates the full training gradient, but on these synthetic problems it reaches objective values much closer to the SAGA reference. This is mathematically consistent with stochastic approximation: a one-sample subgradient has higher variance, so a fixed budget of 1000 iterations is not expected to match a deterministic full-gradient method unless the cheaper iterations are numerous enough or the step schedule is better tuned.

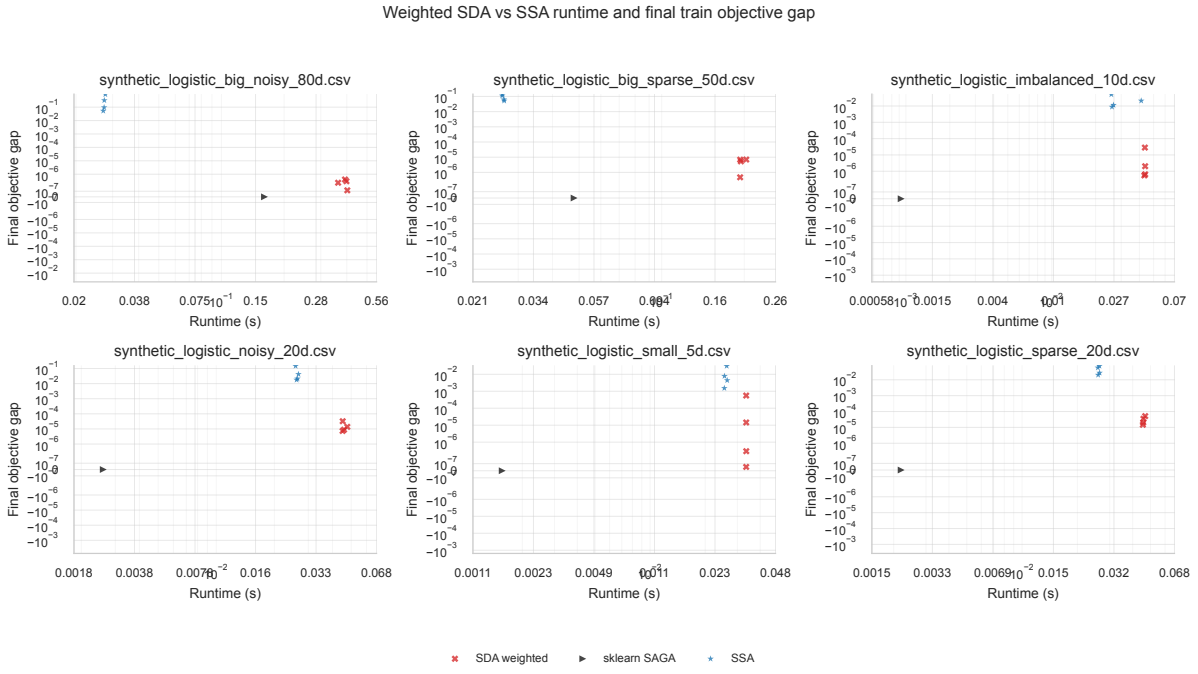

Figure 17: Runtime versus final train-objective gap for weighted SDA, SSA, and sklearn SAGA on synthetic lasso datasets.

Figure 18 shows the γ_{mult} sweep. The weighted SDA curves usually descend rapidly toward the SAGA reference, while SSA curves are noisier and flatten at larger train-objective gaps

under the current iteration budget. The figure uses a logarithmic y-axis because the relevant differences are multiplicative: train-objective gaps near 10^{-6} and 10^{-2} represent qualitatively different optimization accuracy.

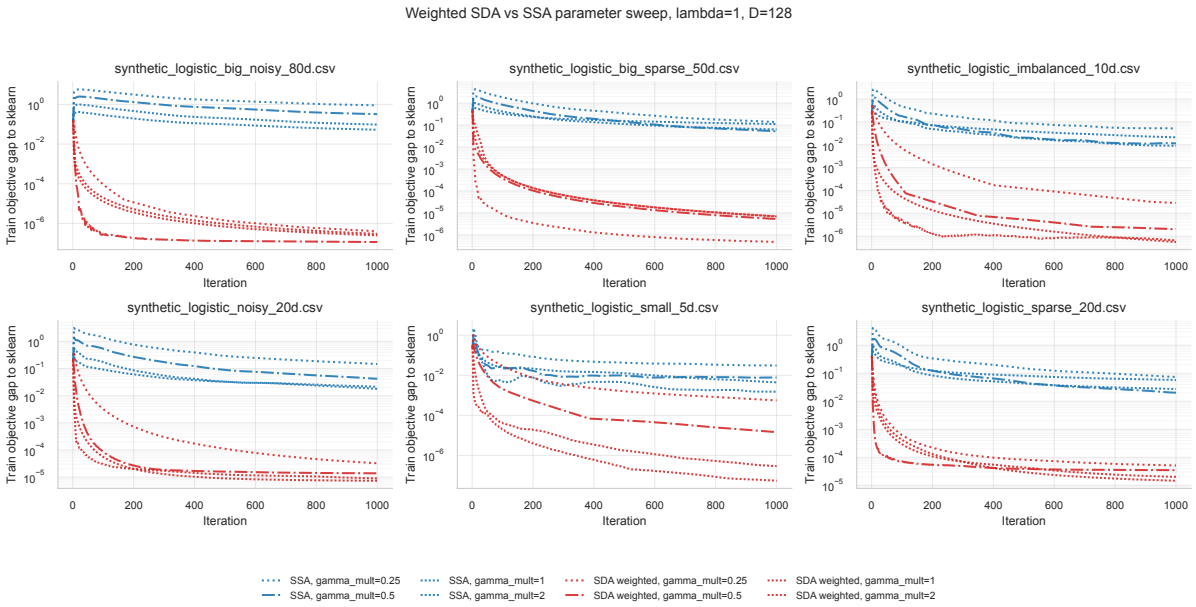

Figure 18: Step multiplier sweep for weighted SDA and SSA on synthetic lasso datasets.

The absolute objective plot in Figure 19 is a useful check on the train-objective-gap plot. The train objectives remain positive and are shown on a logarithmic scale; the SAGA reference line indicates the empirical target. On the large synthetic datasets, the SSA trajectory still improves the objective but does not close the train-objective gap to the same degree as weighted SDA. Thus, for the current implementation and budget, stochastic sampling reduces wall-clock cost but does not compensate for the loss of full-gradient information.

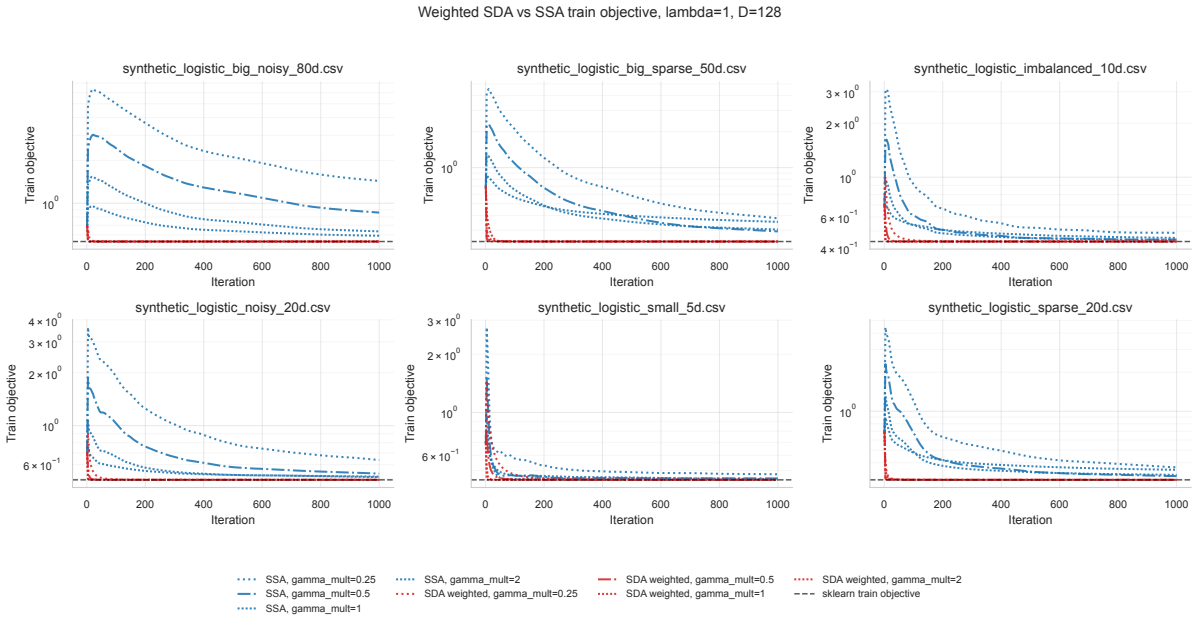

Figure 19: Absolute train objective values for weighted SDA and SSA, with matched sklearn SAGA references.

3.6 Cross-Experiment Discussion

Across the benchmark experiments, the plotted objective gaps are true objective gaps to known optima. Across the lasso experiments, the plotted gaps are empirical train-objective gaps to sklearn SAGA. These two types of objective gap should not be conflated. A true benchmark objective gap measures mathematical suboptimality for the specified objective, whereas a lasso train-objective gap measures agreement with a strong numerical reference.

Several conclusions are stable across the experiments. First, the prox-radius parameter D and step multiplier γ_{mult} materially affect SDA behavior; the experiments do not support a single universally best setting. This is consistent with Nesterov's theory: D is part of the certificate condition, and $\gamma^\star = L/\sqrt{2D}$ only balances a worst-case upper bound. Second, weighted averaging is often helpful, especially in the lasso runs, but it is not uniformly best on every nonsmooth benchmark or every prox geometry. Third, geometry must match the problem: entropy prox is appropriate on the simplex, while diagonal Euclidean prox helps only when its weights reflect the objective anisotropy. Finally, stochastic sampling should be evaluated by runtime and final accuracy together. In the present SSA comparison, cheaper one-sample steps do not reach the same final objective accuracy as weighted full-gradient SDA within the same 1000-iteration budget.

The related literature supports these interpretations but does not prove the specific empirical rankings in this report. Nesterov's primal–dual subgradient analysis justifies the certificate only under the prox-radius condition and gives worst-case black-box rates [1]. Mirror-descent analyses explain why changing the prox geometry can help when it matches the problem structure [4]. Regularized dual averaging gives a stronger specialized treatment of ℓ_1 regularization than the penalty-subgradient implementation used here [2]. SAGA is a specialized finite-sum composite method, so its speed advantage in the lasso experiments is expected rather than surprising [3].

References

[1] Y. Nesterov. Primal-dual subgradient methods for convex problems. *Mathematical Programming*, 120:221–259, 2009.

[2] L. Xiao. Dual averaging methods for regularized stochastic learning and online optimization. *Journal of Machine Learning Research*, 11:2543–2596, 2010.

[3] A. Defazio, F. Bach, and S. Lacoste-Julien. SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives. In *Advances in Neural Information Processing Systems 27*, 2014.

[4] A. Beck and M. Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175, 2003.

[5] K. Koh, S.-J. Kim, and S. Boyd. An interior-point method for large-scale ℓ_1-regularized logistic regression. *Journal of Machine Learning Research*, 8:1519–1555, 2007.